

Apprentissage statistiques : Projet numéro 1

11/04/2026

UE : Apprentissage statistiques

Rabehi Milhane

Djennad Mahdi

Professeur : EL METHNI Jonathan

Sommaire :

1) INTRODUCTION	2
2) ANALYSE SEPARÉE DES IMAGES	2
2.A) IMAGE « STAR WARS » : COMPLEXITÉ ET DÉTAILS	2
2.B) IMAGE « IUT 50 ANS » : LA FORCE DE GRAPHISME.....	3
3) COMPARAISON DES COULEURS POUR $K=3$	5
4) ANALYSE JOINTE ET TRANSFERT DE CENTRES	5
4.A) CLASSIFICATION SIMULTANÉE	5
4.B) TRANSFERT DE CENTRES (CROSS-SEGMENTATION)	5
5) CONCLUSION	6
6) ANNEXE	7

1) Introduction

La segmentation d'image est une technique consistant à regrouper les pixels d'un support visuel en différentes classes selon leurs caractéristiques colorimétriques. Dans le cadre de ce projet, nous utilisons l'algorithme de classification non supervisée K-means pour segmenter deux images distinctes : *StarWars.png* (une scène complexe avec des personnages) et *IUT50ans.png* (un logo graphique).

Chaque pixel est défini par ses trois composantes RGB (Rouge, Vert, Bleu) codées entre 0 et 1. L'objectif est de simplifier ces images en remplaçant chaque pixel par la couleur moyenne (le centroïde) de sa classe. Nous étudierons l'influence du paramètre k (de 2 à 5) sur la qualité de cette simplification et analyserons la possibilité de transférer des centres de couleurs d'une image à une autre. x



Figure 2 : Image originale StarWars utilisée pour la segmentation par k-means



Figure 1 : Image originale IUT50ans utilisée pour la segmentation par k-means

2) Analyse séparée des images

2.a) Image « star wars » : Complexité et détails

Pour cette image, nous avons testé plusieurs valeurs de k pour observer la dégradation de l'information :

- **$k = 2$** : La segmentation est beaucoup trop grossière. On ne dispose que de deux couleurs, ce qui fait disparaître les visages et rend les personnages très difficiles à distinguer du fond.



Figure 3 : Résultat de la segmentation de l'image StarWars avec $k = 2$

- **k = 3** : C'est le seuil de lisibilité. On commence à voir apparaître les grandes zones (les personnages se détachent du fond plus clair), mais les nuances de couleurs sont encore trop pauvres pour apprécier les détails.



Figure 4 : Résultat de la segmentation de l'image StarWars avec $k = 3$

- **k = 4** : Pour nous, il s'agit du **meilleur compromis**. Les détails deviennent nettement plus visibles, les objets sont bien séparés et l'image reste très fidèle à l'originale malgré la forte compression.



Figure 5 : Résultat de la segmentation de l'image StarWars avec $k = 4$

- **k = 5** : Bien que l'on gagne encore en précision, l'amélioration visuelle par rapport à $k = 4$ est assez marginale. Nous estimons que l'augmentation de la complexité n'est pas forcément justifiée ici.



Figure 6 : Résultat de la segmentation de l'image StarWars avec $k = 5$

Pour StarWars, nous préconisons **k = 4** afin de conserver une structure cohérente des personnages

2.b) Image « IUT 50 ans » : La force de graphisme

Le logo présente une structure très différente, avec des aplats de couleurs nets :

- **k = 2** : Le résultat est décevant car le logo devient presque monochrome. On perd immédiatement l'identité visuelle (orange, bleu, rose) au profit d'une teinte unique.



Figure 7 : Résultat de la segmentation de l'image IUT50ans avec $k = 2$

- **k = 3** : Ici, $k=3$ fonctionne déjà très bien. On identifie les composantes principales et le texte reste parfaitement lisible.



Figure 8 : Résultat de la segmentation de l'image IUT50ans avec $k = 3$

- **k = 4** : C'est la valeur **optimale** pour cette image. Elle permet de séparer proprement le bleu, l'orange et les nuances de rose du fond blanc. La segmentation est alors très fidèle au logo d'origine.



Figure 9 : Résultat de la segmentation de l'image IUT50ans avec $k = 4$

Pour le logo IUT, nous recommandons également **k = 4** pour respecter la palette graphique initiale.

3) Comparaison des couleurs pour $k=3$

Afin de mieux comprendre le comportement de l'algorithme, nous avons comparé les centres (les couleurs moyennes) obtenus pour chaque image avec un paramètre $k=3$. Cette comparaison révèle des stratégies d'adaptation très différentes selon la nature du support :

- **Pour StarWars** : Les trois centres identifiés correspondent essentiellement à des niveaux de luminosité : un ton sombre pour les ombres, un intermédiaire pour les visages et vêtements, et un ton clair pour le Stormtrooper et le fond. L'algorithme privilégie ici le contraste pour maintenir la lisibilité des formes.
- **Pour IUT 50 ans** : Les centres capturent directement l'identité visuelle du logo, à savoir le blanc du fond, l'orange et le bleu. Trois classes suffisent ici à restituer l'essentiel de l'information car les couleurs d'origine sont déjà organisées en aplats nets.

4) Analyse jointe et transfert de centres

4.a) Classification simultanée

Nous avons réalisé une analyse "jointe" en fusionnant l'ensemble des pixels des deux images avant de lancer l'algorithme k-means.

- **Résultat** : Les centres obtenus (sombre, clair et intermédiaire) représentent une moyenne globale qui tente de satisfaire les deux univers.
- **Observation** : Cette approche est moins précise pour chaque image prise individuellement. Les couleurs vives du logo se retrouvent atténuées pour s'ajuster aux teintes plus naturelles de StarWars.

4.b) Transfert de centres (Cross-segmentation)

Nous avons enfin testé s'il était possible d'utiliser les centres calculés sur une image pour segmenter l'autre:

- **Du logo vers StarWars** : L'image StarWars prend un aspect très artificiel. Les tons chair et gris sont forcés vers le bleu ou l'orange du logo, créant un rendu proche du "Pop Art" peu fidèle à la réalité.
- **De StarWars vers le logo** : Le logo perd tout son éclat. Les couleurs vives sont remplacées par des variantes de gris et de beige, faisant disparaître son identité graphique.

Enseignement : Ces tests confirment que les centres de gravité sont intrinsèquement liés à la distribution colorimétrique propre de chaque image. Un transfert direct n'est donc pas recommandé pour obtenir une segmentation de qualité.

5) Conclusion

Ce projet met en évidence la flexibilité de l'algorithme k-means pour la segmentation d'images. Nous avons pu observer qu'un support complexe comme une photographie nécessite un k plus élevé ($k=4$) pour rester fidèle, alors qu'un support graphique simple peut se contenter de $k=3$. Enfin, l'importance du choix initial des données est démontrée par l'échec des analyses jointes et des transferts de centres, soulignant que la segmentation doit être spécifiquement calibrée pour chaque image.

6) Annexe

```
#####Faire k-means sur l'une des deux images#####
library(png)
# Lire l'image l'une apres l'autre
#img = readPNG("IUT50ans.png")
img = readPNG("StarWars.png")

# Dimensions de l'image
# dim(img)[1] = nombre de lignes
# dim(img)[2] = nombre de colonnes
# dim(img)[3] = nombre de couleur (RGB = 3)
dim(img)

# Transformer en matrice : une ligne = un pixel, 3 colonnes = RGB
X = matrix(img, ncol = 3)

# Dimensions de la matrice obtenu
dim(X)

# k-means avec k = 2 (la même procédure a été répétée pour k = 3, 4 et 5)
# centers = nombre de clusters
# nstart = 10 permet de relancer l'algorithme 10 fois
res = kmeans(X, centers = 2, nstart = 10)

# Remplacer chaque pixel par la couleur du centroïde de son cluster
# res$cluster donne le numéro de cluster de chaque pixel
# res$centers contient les couleurs moyennes des clusters
segmented = res$centers[res$cluster, ]

# Remettre sous forme originale de l'image
segmented_img = array(segmented, dim = dim(img))

# Afficher
plot.new()
rasterImage(segmented_img, 0, 0, 1, 1)
title("Image segmentée avec k = 2")

#####Préparer la comparaison des deux images pour k = 3#####
# Lire les deux images
img1 = readPNG("StarWars.png")
img2 = readPNG("IUT50ans.png")

# Transformer en matrices pixels x RGB
X1 = matrix(img1, ncol = 3)
X2 = matrix(img2, ncol = 3)
```



```

# k-means séparément sur chaque image avec k = 3
res_starwars = kmeans(X1, centers = 3, nstart = 10)
res_iut = kmeans(X2, centers = 3, nstart = 10)

# Afficher les centres (couleurs moyennes) obtenus pour chaque image
res_starwars$centers
res_iut$centers

# Optionnel : affichage en RGB sur 255, plus lisible
res_starwars$centers * 255
res_iut$centers * 255

#####Faire k-means sur les deux images ensemble#####
# Lire les deux images
img1 = readPNG("StarWars.png")
img2 = readPNG("IUT50ans.png")

# Transformer en matrices pixels x RGB
X1 = matrix(img1, ncol = 3)
X2 = matrix(img2, ncol = 3)

# Fusionner les pixels des deux images
# On obtient ainsi un grand ensemble de pixels contenant
# tous les pixels de StarWars et de IUT50ans
X_total = rbind(X1, X2)

# Appliquer k-means sur les deux images ensemble
res_total = kmeans(X_total, centers = 3, nstart = 10)

# Afficher les centres communs obtenus
# Chaque ligne correspond à une couleur moyenne RGB
res_total$centers

#####Reconstruire les images avec ces centres communs#####
# Nombre de pixels dans chaque image
n1 = nrow(X1)
n2 = nrow(X2)

# Clusters obtenus sur les deux images
clusters_total = res_total$cluster

# Séparer les clusters pour chaque image
# Les n1 premiers pixels correspondent à img1
# Les n2 pixels suivants correspondent à img2
clusters_img1 = clusters_total[1:n1]
clusters_img2 = clusters_total[(n1+1):(n1+n2)]

# Remplacer chaque pixel par son centroïde

```

```

seg_img1 = res_total$centers[clusters_img1, ]
seg_img2 = res_total$centers[clusters_img2, ]

# Reformater en images
seg_img1 = array(seg_img1, dim = dim(img1))
seg_img2 = array(seg_img2, dim = dim(img2))

# Affichage StarWars segmentée avec les centres communs
plot.new()
rasterImage(seg_img1, 0, 0, 1, 1)
title("StarWars avec centres communs")

# Affichage IUT50ans segmentée avec les centres communs
plot.new()
rasterImage(seg_img2, 0, 0, 1, 1)
title("IUT50ans avec centres communs")

#####Utiliser les centres d'une image pour segmenter l'autre#####
#Apprendre les centres uniquement sur l'image StarWars
res1 = kmeans(X1, centers = 3, nstart = 10)
centers_starwars = res1$centers

#Fonction pour affecter chaque pixel au centre le plus proche
assign_cluster = function(X, centers) {

  # Matrice des distances :
  # lignes = pixels
  # colonnes = centres
  dist_matrix = matrix(0, nrow = nrow(X), ncol = nrow(centers))

  # Pour chaque centre, calculer la distance entre ce centre
  # et tous les pixels de l'image
  for (i in 1:nrow(centers)) {
    dist_matrix[, i] = rowSums((X - matrix(centers[i, ],
                                          nrow = nrow(X),
                                          ncol = 3,
                                          byrow = TRUE))^2)
  }
  # Attribuer à chaque pixel le centre le plus proche
  apply(dist_matrix, 1, which.min)
}

# Utiliser les centres appris sur StarWars
# pour segmenter l'image IUT50ans
clusters_img2 = assign_cluster(X2, centers_starwars)

# Remplacer chaque pixel de IUT50ans
# par le centre StarWars le plus proche

```

```

seg_img2_from_img1 = centers_starwars[clusters_img2, ]

# Remettre sous forme d'image
seg_img2_from_img1 = array(seg_img2_from_img1, dim = dim(img2))

# Afficher le résultat
plot.new()
rasterImage(seg_img2_from_img1, 0, 0, 1, 1)
title("IUT50ans avec centres StarWars")

# Apprendre les centres uniquement sur l'image IUT50ans
res2 = kmeans(X2, centers = 3, nstart = 10)
centers_iut = res2$centers

# Utiliser les centres appris sur IUT50ans
# pour segmenter l'image StarWars
clusters_img1 = assign_cluster(X1, centers_iut)

# Remplacer chaque pixel de StarWars
# par le centre IUT50ans le plus proche
seg_img1_from_img2 = centers_iut[clusters_img1, ]

# Remettre sous forme d'image
seg_img1_from_img2 = array(seg_img1_from_img2, dim = dim(img1))

# Afficher le résultat
plot.new()
rasterImage(seg_img1_from_img2, 0, 0, 1, 1)
title("StarWars avec centres IUT50ans")

```